



Enably: how do we build it? break this down

Enably should be built as a **GTM operating system**, not as a generic AI chat app. The core loop is simple: a team answers a guided set of business questions, Enably turns those inputs into editable GTM assets, then uses those approved assets to power messaging, research, and eventually CRM workflows.

Your existing materials already point to the right wedge: **get a founder or SDR from “I have a product” to a usable ICP, sales playbook, and outbound sequence in one session**. Build that first; do not start with broad automation, every integration, or an unconstrained “agent.” ^{[1] [2] [3]}

1. Product in plain English

Enably = the system that creates, stores, and operationalizes a company’s GTM brain.

A new user should be able to:

1. Create a workspace for their company.
2. Complete a guided GTM intake.
3. Receive structured, editable assets:
 - ICP
 - Personas
 - USPs
 - Use cases
 - Positioning and messaging pillars
 - Sales playbook
 - Email/LinkedIn/call sequences
4. Select a company or lead and produce researched, on-brand outreach.
5. Export or eventually sync the approved output into HubSpot.

The differentiation is not merely “generate an email.” It is that emails, scripts, and cadences come from a persistent, structured GTM foundation rather than a blank prompt every time. The original product documents consistently define this as a memory-aware system spanning ICPs, personas, USPs, playbooks, messaging, research, a document hub, and CRM sync. ^[1]

2. Narrow the MVP

The scope in the documents is strong, but it is too large to build all at once. Your MVP needs one promise:

“Create your GTM foundation and first outbound campaign in under 30 minutes.”

Build now vs. later

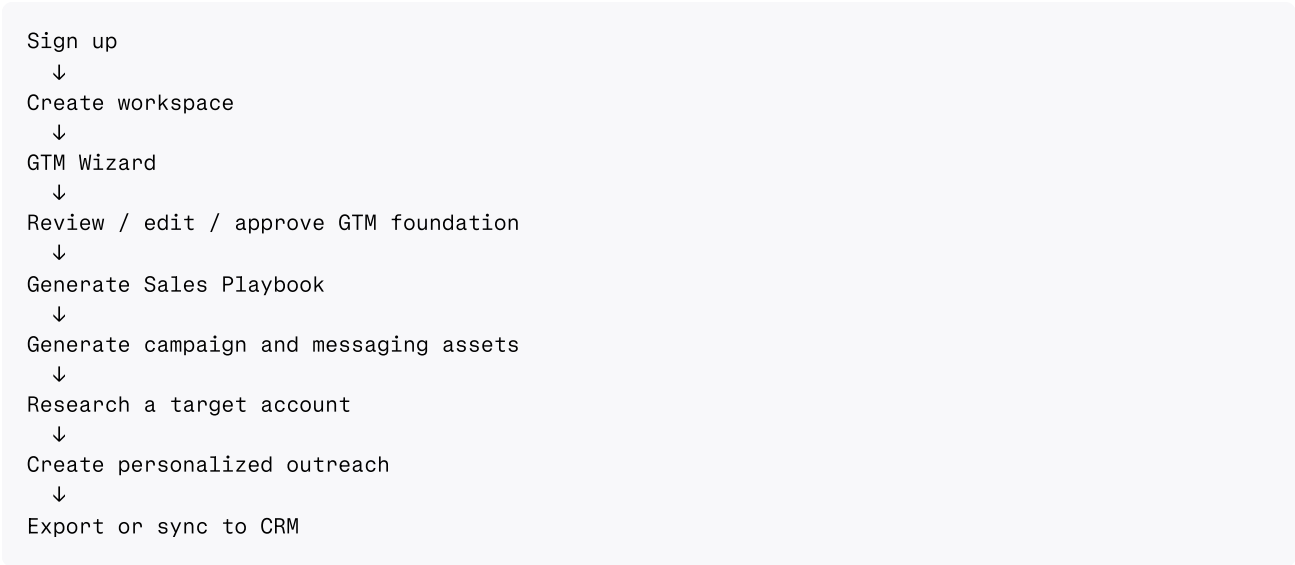
Area	V1: Build now	Later: Do not build first
Authentication	Email login, workspace creation, memberships	Enterprise SSO, complex role configurations
GTM strategy	Guided GTM Wizard producing ICP, persona, USP, use case, positioning	Fully autonomous strategy consultant
Assets	Editable, versioned GTM documents	Full Google Docs/Notion replacement
Messaging	Generate emails, LinkedIn DMs, call opener, 5-step cadence from approved assets	Sending campaigns or managing deliverability
Research	Manual URL/domain input → lightweight company snapshot and outreach angles	Continuous web monitoring, broad enrichment marketplace
Assistant	“Ask about this workspace” with cited internal assets	A universal agent with arbitrary external actions
CRM	Export copy/CSV; optionally HubSpot OAuth + read-only/import path	Two-way sync for every CRM object, writing/sending automatically
Analytics	Activation funnel and generation/export events	Full sales analytics suite and attribution platform

This keeps Enable focused on the **strategy-to-execution handoff**: the place between “we need go-to-market” and “the SDR knows exactly what to say Monday morning.”

The original V1 definition emphasizes a GTM wizard, sales playbook, messaging workshop, research/enrichment, document hub, HubSpot, and a RAG assistant; the strongest build order is to make the first four work together before expanding into integrations. ^[1]

3. The product workflow

The primary user journey



The most important feature: GTM Wizard

Do not make this a long questionnaire. Make it a structured conversation with progressive steps and the ability to skip or revise.

Wizard step	Inputs collected	Asset produced
Company	Company name, URL, category, product, pricing, stage	Company profile

Wizard step	Inputs collected	Asset produced
Product	Problem solved, capabilities, proof, alternatives	Product narrative
Market	Segments, geography, company size, verticals	Market hypotheses
ICP	Firmographics, buying signals, exclusions, current customers	ICP document
Personas	Buyers, champions, blockers, KPIs, pains, objections	Persona cards
Differentiation	Competitors, substitutes, claims, proof points	USPs and positioning
Use cases	Trigger, problem, desired outcome, workflow	Prioritized use-case library
Outreach	Channels, sales motion, ACV, sales cycle, tone	Messaging framework and campaign brief
Review	Structured output review and edits	Approved GTM foundation

Key rule: generate structured JSON first, then render the polished document UI. Do not let the model return only a blob of Markdown. Structured data makes the asset editable, versionable, reusable in downstream prompts, and reliable for retrieval.

Example: the action loop

A founder says:

“We sell AI-powered security questionnaires to mid-market SaaS companies.”

Enably should derive and show—not silently assume:

- **ICP:** B2B SaaS, 100–1,000 employees, security/compliance pressure, multiple enterprise deals.
- **Economic buyer:** VP Security, CISO, Head of GRC.
- **Champion:** Sales Engineering or RevOps leader dealing with slow questionnaires.
- **Trigger:** Enterprise deal delayed by a security review.
- **Value proposition:** Complete security questionnaires faster while preserving approved answers and governance.
- **Campaign:** Five touches across email, LinkedIn, and call tasks.
- **Personalization:** “Noticed you’re hiring for GRC / recently announced enterprise expansion / added a compliance role.”

The user must be able to edit and approve each item before that data becomes their workspace truth.

4. Agent architecture

Do **not** begin with a swarm of autonomous agents. Start with deterministic workflows plus a small set of tightly scoped generation services.

Recommended agent system

Service / agent	Responsibility	Inputs	Output	Permission
GTM Intake Compiler	Turns wizard responses into a clean GTM brief	Wizard answers, uploads, company URL	Validated GTM brief JSON	Read workspace data
ICP Generator	Produces ICP and exclusion criteria	GTM brief	ICP JSON/document	Create draft only
Persona Generator	Creates buyer, champion, user, blocker personas	ICP, product narrative	Persona set	Create draft only

Service / agent	Responsibility	Inputs	Output	Permission
Positioning Generator	Produces USPs, differentiators, proof gaps	GTM brief, ICP, personas	Messaging architecture	Create draft only
Playbook Generator	Creates targets, workflows, qualification, cadences	Approved strategy assets	Sales playbook	Create draft only
Messaging Generator	Makes channel-specific outreach	Approved assets + lead context	Email, DM, call script, sequence	Create draft only
Research Agent	Extracts relevant public facts from a user-supplied domain/URL	URL/domain and target context	Research snapshot with sources	Read public web only
Workspace Assistant	Answers questions with internal asset retrieval	User prompt, approved workspace context	Cited answer / proposed draft	No external writes
Integration Worker	Handles OAuth tokens, sync jobs, exports	User-approved integration request	Sync status and mapped records	Writes only after approval

The core orchestration policy

```
User input
  → validate and normalize
  → save source-of-truth input
  → generate structured draft
  → validate output against schema
  → show user for edit/approval
  → mark version as approved
  → index approved version for assistant/messaging
  → allow exports or integration actions
```

That approval step is important. You do not want an assistant making CRM suggestions from an unapproved, hallucinated persona or an outdated positioning document.

Your project’s ROSTR design principles fit Enably well:

- **PAL:** interpret messy founder input and identify missing information.
- **JTBD:** turn product descriptions into desired buyer outcomes, pains, and progress.
- **RAG DAL:** research only decision-relevant items and retain source provenance.
- **NPAO:** sequence work—foundation first, then playbook, then messaging, then integrations.

This makes Enably a GTM-specialized implementation of the ROSTR approach rather than a disconnected collection of prompt buttons. [2] [1]

5. Technical architecture

Recommended stack

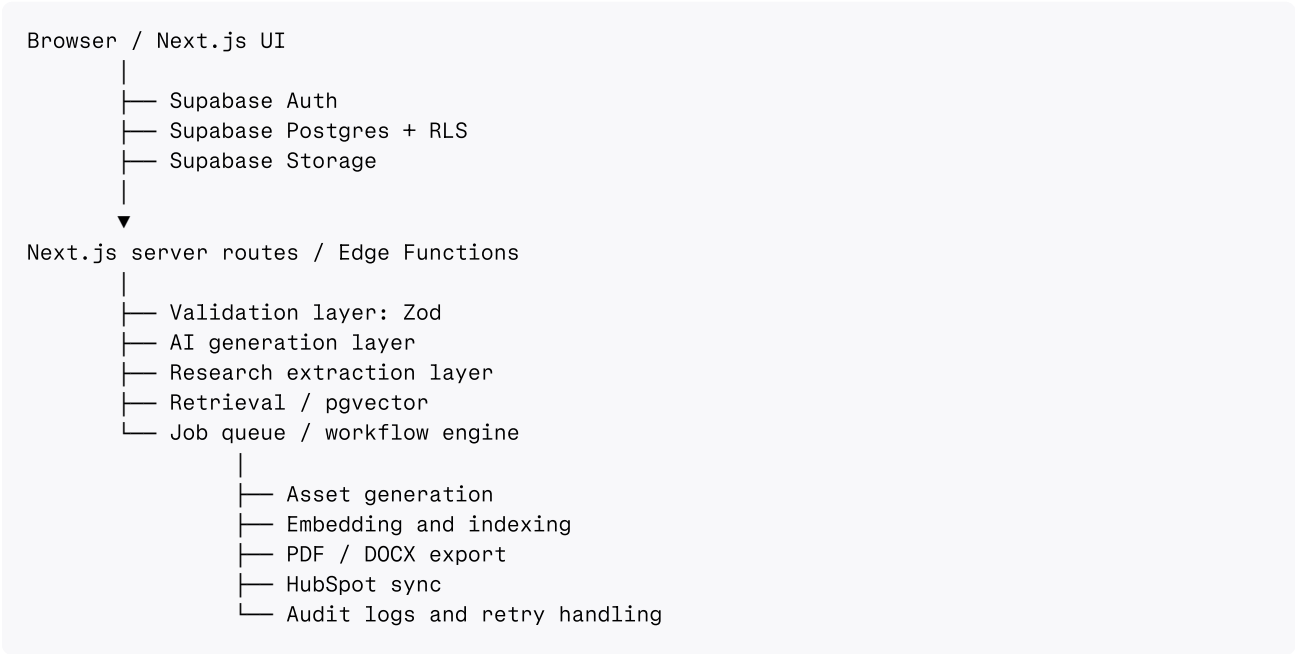
Keep it lean and cohesive:

Layer	Recommendation	Why
Web app	Next.js App Router + TypeScript	Fast product iteration, server actions/API routes, strong SaaS ecosystem
UI	Tailwind + shadcn/ui + Radix primitives	Matches your desired modern, accessible SaaS aesthetic
Auth	Supabase Auth	Workspace-aware authentication and low operational overhead
Database	Supabase Postgres	Multi-tenant relational data, auditability, SQL control

Layer	Recommendation	Why
Authorization	Postgres RLS from day one	Prevents workspace data leakage
File storage	Supabase Storage	Secure uploads and asset exports
AI gateway	Server-side API routes or Edge Functions	Keep provider keys out of the browser
Model workflow	Structured outputs + Zod schemas	Reliable, editable asset generation
Vector search	pgvector in Supabase	Retrieval over approved GTM documents
Background jobs	Inngest or Trigger.dev	Generation, indexing, exports, CRM sync retries
Observability	Sentry + PostHog	Errors, product activation, feature usage
Hosting	Vercel + Supabase	Fast V1 deployment
CRM first	HubSpot OAuth	Your docs designate HubSpot as the primary initial integration

The previously drafted technical direction—Next.js, Tailwind, shadcn/ui, Supabase, pgvector, Supabase Auth/Storage/functions, AI generation, and PDF output—is appropriate, with one adjustment: **avoid Prisma at first unless you truly need it**. Supabase’s generated TypeScript types plus SQL migrations are simpler for a solo/lean build. Add an ORM later only if the team and backend complexity justify it. ^[1]

System diagram



Core data model

Use a durable multi-tenant object model from day one:

```
users
workspaces
workspace_members

companies
products
gtm_briefs

documents
document_versions
```

```
document_sections
document_sources
document_embeddings

icps
personas
usps
use_cases
playbooks
templates
campaigns

research_snapshots
research_sources
leads

conversations
messages
generation_runs
prompt_versions

integrations
integration_connections
sync_jobs
audit_events
```

Every business object should include at minimum:

```
id
workspace_id
created_by
created_at
updated_at
status
version
source_document_id
approval_state
```

Two essential tables

documents

```
type Document = {
  id: string
  workspaceId: string
  type:
    | "gtm_brief"
    | "icp"
    | "persona"
    | "usp"
    | "use_case"
    | "positioning"
    | "playbook"
    | "message_template"
    | "research_snapshot"
  title: string
  status: "draft" | "approved" | "archived"
  currentVersionId: string
  createdBy: string
  createdAt: string
```

```
    updatedAt: string
  }
```

document_versions

```
type DocumentVersion = {
  id: string
  documentId: string
  versionNumber: number
  contentJson: Record<string, unknown>
  renderedMarkdown: string
  generationRunId?: string
  createdBy: string
  approvalState: "draft" | "approved" | "rejected"
  approvedBy?: string
  approvedAt?: string
  createdAt: string
}
```

This is the backbone of Enably. Once this works, the assistant, messaging, exports, and CRM sync can all depend on the same approved GTM knowledge rather than creating parallel, inconsistent data.

6. Build sequence

Sprint 0: Product decisions — 2–3 days

Before writing features, lock these choices:

- Primary persona: **founder / Head of Growth** at an early-stage B2B company.
- Core first-session outcome: **approved ICP + first outbound sequence**.
- First integration: **HubSpot** only, and make it optional.
- Approval policy: outputs are drafts until explicitly approved.
- Pricing assumption: start with a free demo or pilot workspace; do not build complicated billing before users validate the core workflow.
- Design system: electric indigo #635BFF, emerald success state #10B981, light SaaS interface, rounded 2xl cards—as defined in your GTM overview.^[1]

Deliverables

- One-page product brief.
- User journey map.
- Clickable wireframe.
- JSON schemas for initial asset types.
- Initial Supabase schema and RLS plan.
- Event taxonomy.

Sprint 1: Working app shell — 1 week

Build the product skeleton with no real AI complexity yet.

Build

- Next.js application.

- Supabase project, migrations, RLS policies.
- Authentication: sign-up, login, logout.
- Workspace creation and member model.
- Application shell: sidebar, dashboard, page routing.
- Basic pages: Dashboard, Wizard, Hub, Messaging, Settings.
- Empty states, loading states, error states.
- Event instrumentation.

Definition of done

- A user can create a workspace.
- User only sees records belonging to their workspace.
- The interface is responsive and accessible.
- The dashboard accurately shows “no GTM assets yet.”

Sprint 2: GTM Wizard and document engine — 1-2 weeks

This is the MVP’s heart.

Build

- Multi-step wizard built with React Hook Form and Zod.
- Autosave and resume later.
- `gtm_brief` JSON schema.
- AI endpoint that accepts validated inputs.
- Structured generation for:
 - ICP
 - Persona set
 - USPs
 - Use cases
 - Positioning statement
- Document detail screens with editing.
- Version history.
- “Approve for use” control.
- Export to Markdown and PDF.

Definition of done

- A new workspace can complete the flow.
- Enably returns usable, structured GTM assets.
- User can edit before approval.
- Approved assets remain available to downstream features.
- Generation failures do not lose user inputs.

Sprint 3: Sales playbook and messaging — 1 week

Turn strategy into execution.

Build

- Playbook generator:
 - Qualification method or custom criteria.
 - Activity targets.
 - Sales stages.
 - SOPs.
 - Objection handling.
 - Cadence structure.
- Messaging Workshop:
 - Email generator.
 - LinkedIn DM generator.
 - Call opener/call script generator.
 - Five-step outbound cadence.
- Variable system:
 - {{first_name}}
 - {{company_name}}
 - {{pain_point}}
 - {{trigger}}
 - {{use_case}}
- Copy, save, duplicate, export.

Your spreadsheet already identifies the initial functional tools: **Call Script Generator, Email Generator, ICP Creator, User Persona Creator, and Sales Playbook Generator**. Those should become the first structured workflows—not five unrelated prompt screens.^[3]

Definition of done

- A user can generate an email or call script from an approved ICP/persona.
- Templates include clear provenance such as “built from ICP v2 and Persona: VP Revenue.”
- Users can modify and save templates.

Sprint 4: Research and contextual messaging — 1 week

This makes Enably feel intelligent without requiring full-blown autonomous prospecting.

Build

- Domain/URL intake.
- Public-page extraction.
- Company snapshot:
 - What the company does.
 - Likely customer.

- Recent positioning or product clues.
- Potential GTM trigger.
- Outreach angle.
- Source URLs and captured date.
- “Draft email from this research” action.
- Human-review warning when evidence is sparse.

Guardrail

Never present unverified inference as fact. Separate:

```
Observed facts
Likely hypotheses
Suggested outreach angle
Missing information
```

This directly supports your proposed “research micro-agent” while preserving the source-aware behavior required for a credible GTM product. ^[1]

Sprint 5: Assistant and retrieval — 1 week

Only now build chat.

Build

- Workspace assistant UI.
- Retrieval limited to approved documents, selected research snapshots, and current lead context.
- Citations linking to internal documents and source snippets.
- Context picker:
 - Workspace GTM foundation.
 - Current ICP/persona.
 - Sales playbook.
 - Specific target account.
- Suggested actions:
 - “Draft a cadence.”
 - “Summarize our ICP.”
 - “Write an objection response.”
 - “Find gaps in our positioning.”

Do not do yet

- “Go execute this campaign.”
- Broad web research with no user request.
- CRM writes without review.
- Agents creating arbitrary integrations.

Sprint 6: HubSpot and pilot — 1–2 weeks

HubSpot comes after Enably creates genuinely useful assets internally.

Initial HubSpot scope

- OAuth connection.
- Read companies and contacts selected by the user.
- Map fields for personalization.
- Create a note or draft activity only after explicit confirmation.
- Export contacts/templates to CSV as a fallback.
- Sync status and error log.

Avoid in V1

- Auto-sending.
- Automated bulk writes.
- Bi-directional conflict resolution across all CRM records.
- Salesforce, Pipedrive, and every other CRM simultaneously.

The source plan identifies HubSpot as the initial CRM integration and uses it to synchronize contacts, companies, and notes; make this an approved, observable, retryable workflow rather than a hidden agent action.^[1]

7. Quality and guardrails

Enably will be handling company strategy, customer data, and eventually CRM credentials. The product should act like an assistant that proposes work—not an unreviewed autopilot.

Non-negotiable rules

- **Workspace isolation:** Enforce row-level security on every table that contains customer information.
- **Server-only secrets:** No AI, CRM, storage, or webhook keys in client code.
- **Structured output:** Validate every AI asset against Zod/JSON schemas before saving.
- **Draft-first generation:** AI-generated assets are drafts by default.
- **Explicit approval:** Require user approval before an asset becomes retrieval context or feeds a CRM action.
- **Provenance:** Store prompt version, source inputs, model run metadata, and references used.
- **Research honesty:** Mark assumptions separately from observed facts.
- **Auditable integration writes:** Store who authorized what, when, target object, request payload summary, and response.
- **Rate limits:** Protect generation and research endpoints.
- **PII minimization:** Only collect contact data that is required for the requested task.
- **Reliable jobs:** Make generation and sync operations idempotent; a retry must not duplicate a CRM record or document version.

8. What to measure

Your source plan proposes strong activation benchmarks—first ICP in under 15 minutes, three assets exported on day one, and early CRM-connect adoption. Use them as hypotheses to validate, not marketing claims until real data exists. ^[1]

Initial product metrics

Funnel stage	Metric	Why it matters
Activation	Time to first approved ICP	Core moment of value
Activation	Wizard completion rate	Reveals whether intake is too long or confusing
Value	Assets approved per workspace	Indicates outputs are useful, not merely generated
Value	First cadence generated	Confirms strategy became execution
Engagement	Messaging generated from approved context	Measures use of the GTM brain
Retention	Weekly active workspaces with at least one asset edit or generation	Better than raw logins
Integration	HubSpot connection rate	Reveals readiness for workflow integration
Quality	Regeneration/edit rate per asset type	Finds weak prompts and schemas
Trust	Research outputs with viewed sources	Measures evidence use
Reliability	Generation success rate and p95 completion time	Product credibility

9. Your first build ticket

If you start tomorrow, build only this:

A user signs up, creates a workspace, completes a GTM Wizard, receives an editable ICP, approves it, and generates a first outbound email using that ICP.

That single vertical slice proves the product loop:

Input → GTM intelligence → approval → practical execution

First repository structure

```
apps/web/
  app/
    (marketing)/
    (auth)/
  app/
    dashboard/
    wizard/
    hub/
    messaging/
    settings/
  api/
    generate/
    documents/
    research/
    integrations/
  components/
    wizard/
    documents/
    messaging/
```

```
layout/  
lib/  
  ai/  
  auth/  
  db/  
  schemas/  
  prompts/  
  retrieval/  
supabase/  
  migrations/  
  seed.sql
```

First four API contracts

```
POST /api/generation/gtm-brief  
POST /api/generation/icp  
POST /api/generation/message  
POST /api/documents/:id/approve
```

First schemas to create

```
GtmBriefSchema  
IcpSchema  
PersonaSchema  
PositioningSchema  
MessageTemplateSchema  
DocumentVersionSchema
```

Bottom line

Build Enably as a structured GTM knowledge and execution system:

1. Capture the company’s reality through a guided wizard.
2. Turn it into editable, approved strategic assets.
3. Use those assets to generate better playbooks and outbound messaging.
4. Add targeted research to personalize the work.
5. Add HubSpot only after the internal workflow is useful and trustworthy.
6. Keep every agent constrained, source-aware, versioned, and approval-gated.

That path gives you a focused, testable product in roughly six focused build sprints instead of trying to ship an entire “AI RevOps platform” at once. The product’s clearest wedge is not generic generation—it is a shared, memory-aware GTM foundation that makes every downstream sales activity more consistent. ^[2] ^[3] ^[1]



extract each of these custom gpt files into skills (or agents) full yaml contract, downloadable and installable into any agent harness, full agent set up backend with rostrv2 runtime, etc. and each individual skill file

I compiled the supplied Enably custom-GPT documents into a portable, installable **ROSTRv2 agent package**. It includes the full agent set, individual skill files, a runtime manifest, routing and state-machine policies, schema contracts, approval-gated tool adapters, backend setup documentation, and contract tests.

The package preserves the original GPT responsibilities while converting them from prompt-only assistants into durable, harness-agnostic agents with workspace boundaries, artifact versioning, provenance, structured outputs, and safe external-action controls. It was extracted from the KPI Metric Architect, LinkedIn/Social DM Architect, and Enably Signal product/agent specifications. [\[7\]](#) [\[8\]](#) [\[9\]](#)

Downloadable package

The generated **Enably ROSTRv2 agent pack** is attached to this response as a ZIP archive. It contains 19 installable files.

Package layout

```
enably-rostrv2-agent-pack/
├── README.md
├── .env.example
├── agents/
│   ├── enably-orchestrator.agent.yaml
│   ├── kpi-metric-architect.agent.yaml
│   ├── social-dm-architect.agent.yaml
│   └── sales-ops-copilot.agent.yaml
├── skills/
│   ├── kpi-framework/
│   │   └── SKILL.md
│   ├── social-dm/
│   │   └── SKILL.md
│   ├── gtm-intake/
│   │   └── SKILL.md
│   ├── approval-gating/
│   │   └── SKILL.md
│   └── artifact-versioning/
│       └── SKILL.md
├── contracts/
│   ├── rostrv2.manifest.yaml
│   ├── artifact-envelope.schema.json
│   └── tool-adapters.yaml
├── runtime/
│   ├── routing.yaml
│   └── state-machine.yaml
├── docs/
│   ├── backend-setup.md
│   └── installation.md
└── tests/
    └── contract-checklist.md
```

Included agents

Agent	Extracted function	Production-ready responsibility
Enably Orchestrator	Not explicitly a standalone Custom GPT; derived as the coordinating runtime layer	Compiles a request, resolves intent, routes to the correct specialist, validates artifacts, and enforces run state and approval policy
KPI Metric Architect	KPI Activity Metric Generator	Converts business/team goals into outcome KPIs, leading activity metrics, owners, formulas, cadence, source systems, and dashboard-ready schemas
LinkedIn & Social DM Architect	Social Messaging Architect	Produces platform-aware connection notes, InMails, social DMs, follow-ups, response branches, and CRM-ready template JSON
Enably Signal Sales Ops Copilot	Enably Signal conversational sales-ops setup	Guides an outbound foundation: business/ICP intake, mailbox and deliverability readiness, lead-file quality, Day 1–Day 4 sequence, KPI contract, and daily send-health design

The KPI agent retains the original constraints: pair outcomes with leading indicators, define formulas and owners, tailor work to GTM motion and company stage, limit the default KPI set to roughly three to five per team, and avoid financial forecasting or vanity metrics.^[8]

The social messaging agent retains the original platform guardrails: enforce LinkedIn connection notes under 300 characters, avoid fabricated personalization and sales-heavy cold openers, default to draft-only behavior, and produce human, reply-oriented messaging rather than generic AI copy.^[9]

The Enably Signal agent includes the extracted MVP behavior: chat-led GTM intake, deliverability/DNS/SES readiness, lead-file workflows, short email sequences, research panes, and a daily send-health model—while preventing accidental outbound sends, silent CRM updates, secret exposure, and unsupported research claims.^[7]

Individual skill contracts

Each skill is independently mountable in an agent harness.

kpi-framework

Trigger terms: KPI, metric, dashboard, activity metric, performance indicator, OKR.

Input contract:

```
inputs:
- goal
- team
- company_stage
- gtm_motion
- data_sources
```

Output contract:

```
outputs:
- kpi_framework
- metric_map
- dashboard_schema
```

Behavior:

- Maps strategic goals to a small, decision-useful KPI set.
- Adds controllable weekly inputs that drive outcomes.

- Requires definition, formula, unit, owner, system of record, cadence, target status, and caveat for each metric.
- Labels suggested benchmarks separately from evidence-backed benchmarks.

social-dm

Trigger terms: LinkedIn, connection note, InMail, DM, Twitter/X, social outreach.

Input contract:

```
inputs:
  - platform
  - message_type
  - audience
  - offer
  - personalization_facts
  - tone
```

Output contract:

```
outputs:
  - message_set
  - character_counts
  - sequence_tree
```

Behavior:

- Produces one-shot messages or two-to-five-touch sequences.
- Offers soft and direct CTA variants when useful.
- Includes message character counts.
- Prevents unsupported personal claims and prevents any direct message dispatch.

gtm-intake

Trigger terms: outbound setup, GTM setup, ICP, lead upload, sequence, sales ops.

Input contract:

```
inputs:
  - business
  - product
  - icp
  - goals
  - crm
  - mailbox
  - domain
  - lead_source
```

Output contract:

```
outputs:
  - gtm_brief
  - setup_checklist
  - icp_pack
```

- sequence
- readiness_report

Behavior:

- Captures the GTM operating context before generating messaging.
- Builds ICP and exclusions before sequences.
- Creates a Day 1–Day 4 cadence with reply/bounce suppression logic.
- Defines send-health reporting: sent, delivered, bounced, opened, and replied.
- Treats DNS changes, CRM writes, and sending as approval-gated external actions.

approval-gating

This is the universal write-protection skill.

```
inputs:
  - action
  - resolved_target
  - exact_payload
  - approval_token
outputs:
  - approval_request
  - approved_execution
  - rejection
```

It ensures that the host harness—not conversational language alone—issues approval for an exact, resolved payload. Once approved, agents cannot silently alter recipients, add records, widen audience scope, or modify the payload.

artifact-versioning

This establishes the portable output contract:

```
required_metadata:
  - artifact_id
  - artifact_type
  - workspace_id
  - version
  - status
  - owner_agent
  - timestamps
  - provenance
  - approval_state
```

Drafts may change. Approved artifacts are immutable; revisions create a new version rather than overwriting the original.

ROSTRv2 runtime contract

The root manifest is:

```
contracts/rostrv2.manifest.yaml
```

It declares:

- The Enably orchestrator as the entry agent.
- The specialist agent registry.
- The installable skill registry.
- A common artifact envelope schema.
- Tool adapter names rather than vendor-specific implementations.
- Runtime routing and phase state machine.
- Safety denials for unapproved outbound sends, silent CRM writes, secret exposure, financial actions, and unsupported facts treated as truth.

Routing policy

```
kpi_design / metric_framework / dashboard_schema
  → kpi-metric-architect

linkedin_message / social_dm / inmail / connection_note
  → social-dm-architect

sales_ops_setup / outbound_setup / lead_import / sequence_setup
  → sales-ops-copilot

gtm_intake / icp / persona / positioning / sales_playbook
  → sales-ops-copilot

unclear input
  → enably-orchestrator
```

Runtime lifecycle

```
received
  → normalizing
  → planning
  → generating
  → validating
  → draft_ready
  → approval_required, if an external write is requested
  → approved
  → executing_write
  → completed
```

Alternate state paths cover clarification requests, rejected actions, canceled runs, blocked dependencies, validation remediation, and errors with retry-safe execution.

Backend setup

The package includes `docs/backend-setup.md`, which specifies a practical production backend:

```
Frontend / API
  Next.js or any authenticated web/API layer

ROSTRv2 Runtime
  Orchestrator + specialist registry
  Route policy + state machine
  Structured artifact validation

Persistent state
```

Postgres / Supabase Postgres
workspace state, runs, artifacts, approvals, audits

Storage

Object storage for uploads and exports

Retrieval

Vector-capable index over approved GTM artifacts

Async work

Queue and worker system for generation, research, exports, syncs, retries

Secrets

Server-side secret manager for AI, CRM, AWS, email, and integration credentials

Required backend entities

workspaces
members
runs
artifacts
artifact_versions
approvals
audit_events
tool_executions
integrations
integration_connections
sync_jobs
conversations
messages
retrieval_chunks

Required API endpoints

POST /v1/runs
POST /v1/runs/{id}/input
GET /v1/runs/{id}
POST /v1/approvals
POST /v1/tools/execute
GET /v1/artifacts/{id}

The tool adapter contract is deliberately vendor-neutral, so it can map to n8n, Temporal, Inngest, AWS Lambda, Supabase Edge Functions, LangGraph, PydanticAI, OpenAI Agents SDK, Claude-based subagents, or a custom ROSTR runtime.

Installation model

1. Extract the attached ZIP into your agent repository or workspace.
2. Load `contracts/rostrv2.manifest.yaml` into your agent registry.
3. Register the four `agents/*.agent.yaml` files.
4. Mount each `skills/*/SKILL.md` file in your harness's skill directory.
5. Bind generic tools from `contracts/tool-adapters.yaml` to your own infrastructure:
 - `state.read`
 - `state.write`

- `artifact.store`
- `retrieval.search`
- `web.research`
- `crm.read`
- `crm.write`
- `email.send`
- `social.send`

6. Configure secrets server-side with the supplied `.env.example`.
7. Implement workspace-scoped authorization, approval-token verification, immutable artifact versions, idempotency keys, and audit logs.
8. Run `tests/contract-checklist.md` before enabling CRM writes or outbound sending.

What remains intentionally unbuilt

This package gives you a deployable **agent specification and runtime contract**, not a live integration implementation. The following must be bound to the selected infrastructure:

- Actual LLM provider/model adapter.
- Database and storage provisioning.
- OAuth flows for HubSpot or other CRMs.
- AWS SES, SNS, Bedrock, or equivalent provider bindings.
- Actual DNS automation, if you decide to support it.
- Approved research/search provider.
- Host-harness approval UI and signed token issuance.
- User/workspace authentication and permissions.
- Observability stack, retries, queue workers, rate limits, and billing.

That separation is deliberate: it keeps the agent pack portable across different harnesses while making security and side effects explicit rather than hidden in prompt text.



Needs to be able to connect to your domain provider AWS or godaddy etc and make changes so that MCP creation or composio for integration as well as set up and connect to all your tools to do research read analyze state etc current t state versus desired state analysis

Yes—Enably needs a **connection and state-reconciliation** layer, not just agents that generate documents. It should connect to your domain/DNS provider and business tools, inspect the actual configuration, compare it with your approved desired state, then propose and execute precisely approved changes.

That expands the previous package from “agent specifications” into a connected operations system. The earlier ZIP is a specification scaffold—not a functioning ROSTRv2 backend or universally plug-and-play harness installation.

1. The operating loop

The new core workflow should be:

```
Connect accounts
  → Discover tools and permissions
  → Read current state
  → Collect desired outcomes
  → Research requirements through RAG DAL
  → Compare current vs. desired state
  → Produce a change plan
  → Review exact changes
  → Execute approved actions
  → Verify results
  → Monitor drift
```

This aligns with your existing ROSTR orchestrator: create run state, dispatch bounded work, preserve artifact contracts, and stop when permissions or required inputs are missing. Concrete MCP/API bindings belong in the deployment tool manifest rather than being assumed by an agent. ^[10] ^[11]

For example, the user says:

“Connect my domain and HubSpot, inspect my outbound setup, and get it ready for Enably.”

Enably should first report what it can actually observe:

Area	Current state	Desired state	Proposed action
Domain	Registrar identified; authoritative DNS host unresolved	Confirm DNS control	Inspect nameservers and identify the writable zone
Email authentication	Records and provider status collected	Approved sending configuration	Prepare exact record changes from provider-generated values
CRM	Connection active; field mapping incomplete	Approved lead/contact mapping	Propose field mapping and resolve duplicates
Sequences	Templates exist; reply suppression unverified	Approved sequence with stop conditions	Configure and test suppression
Reporting	Delivery events available; replies not connected	Clearly defined send-health dashboard	Connect missing event sources

This is a sample report structure—not a diagnosis of your actual accounts.

2. Integration architecture

Use three interchangeable connection paths rather than forcing everything through one vendor.

Connection path	Proposed use	Boundary
Composio	Supported SaaS authentication, tool discovery, and actions	Verify the actual toolkit, available actions, and granted scopes
Existing MCP server	Tools that already expose a suitable MCP interface	Inspect schemas and enforce Enably policy around each call
Custom API adapter/MCP server	Unsupported providers, specialized operations, or stricter controls	Build and test a provider-specific adapter before exposing it

Composio supports custom tools alongside its remote tools and authenticated toolkit extensions. It can therefore serve as an integration layer, while Enably retains its own planning, permissions, and approval

logic. Do not assume that every requested provider or operation is available just because Composio is connected. ^[12]

DNS provider connections

AWS Route 53: build a provider adapter for record discovery and approved change batches. Route 53 supports CREATE, UPSERT, and DELETE; its change batches apply transactionally within the request. IAM can restrict writes by hosted zone, record name, record type, and action. ^{[13] [14]}

GoDaddy: build a separate adapter and verify authentication and DNS API access during onboarding. GoDaddy announced single-domain DNS API eligibility in April 2026; its newer documentation distinguishes scoped personal access tokens from classic developer keys for some APIs. Select credentials according to the actual endpoint rather than hardcoding one universal authentication method. ^{[15] [16]}

Important design distinction: **the registrar and authoritative DNS host may be different**. Enably should identify both before choosing where to write records; connecting the registrar alone must not count as verified DNS control.

MCP creation workflow

For unsupported tools, add an Integration Builder that produces:

- 1. A provider capability manifest and authentication specification.
- 2. Read/write tools with explicit input and output schemas.
- 3. A provider adapter with pagination, rate-limit handling, timeouts, and error normalization.
- 4. An MCP wrapper exposing only approved capabilities.
- 5. Tests for authentication, schema validation, tenant isolation, and denied writes.
- 6. A deployment proposal requiring approval before installation or production changes.

Generating an MCP server should not automatically grant it credentials or production access.

3. Expanded agent set

Keep the existing GTM specialists, but add a dedicated operational layer.

Agent	Responsibility	Access
Connection Manager	Start account connection, validate identity and scopes, track expired connections	Connection metadata; no arbitrary business writes
Integration Builder	Create adapters, MCP definitions, and installation proposals	Documentation and sandbox development
State Discovery Agent	Inventory DNS, CRM, mailbox, sequence, and reporting configuration	Read-only approved tools
Desired-State Compiler	Turn business intent into measurable requirements and constraints	Workspace context and user-approved objectives
Gap Analyst	Compare observed state with desired state; identify dependencies and blockers	State snapshots and evidence
Domain Operations Agent	Prepare and apply exact DNS changes	Scoped provider access; gated writes
Change Executor	Execute approved plans through allowlisted adapters	Exact approved capabilities only
Verification Agent	Re-read state, test outcomes, and identify drift	Read-only verification tools

The orchestrator coordinates these agents; it should not personally hold unrestricted access to every provider. Your existing ROSTR tool manifest already separates orchestration, repository access, and production

changes, using project-scoped identities and audit logs.^[11]

State must carry evidence

Every observed property should include:

```
observation:
  resource_id: "resolved-provider-resource-id"
  property: "dns.record"
  value: {}
  source_connection_id: "workspace-connection-id"
  observed_at: "timestamp"
  evidence_ref: "snapshot-reference"
  freshness: fresh
  certainty: observed
```

The gap analysis must distinguish:

- Verified mismatch.
- Desired state already satisfied.
- Unknown because access is missing.
- Unknown because data is stale.
- Inference requiring confirmation.
- Conflict requiring a business decision.

“Could not inspect” must never become “not configured.”

4. Runtime contract extension

The following is a **proposed Enably contract**, not a verified native ROSTRv2 schema. The project evidence describes orchestration and permission policies, but does not establish a complete deployable ROSTRv2 API contract.^{[10] [11]}

```
contract_version: "enably.operations.v0.2"
kind: ConnectedOperationsPolicy

connections:
  mechanisms:
    - composio
    - existing_mcp
    - custom_api_adapter
    - custom_mcp
  require:
    - workspace_binding
    - authenticated_account_identity
    - verified_capability_inventory
    - server_side_secret_reference
  default_access: read_only

discovery:
  outputs:
    - connection_inventory
    - resource_inventory
    - current_state_snapshot
    - evidence_ledger
    - access_gaps
  unknown_is_not_absent: true
```

```

analysis:
  inputs:
    - current_state_snapshot
    - approved_desired_state
    - verified_requirements
  outputs:
    - gap_report
    - dependency_graph
    - proposed_change_plan

changes:
  require:
    - resolved_provider_account
    - resolved_resource_target
    - exact_provider_request
    - before_state_snapshot
    - verification_plan
    - rollback_or_recovery_plan
  approval_binding:
    fields:
      - workspace_id
      - connection_id
      - target_resource
      - operation
      - exact_payload_hash
      - before_state_hash
      - expires_at
  reject_on:
    - missing_permission
    - changed_payload
    - expired_approval
    - stale_before_state
    - unexpected_resource_change

execution:
  server_side_policy_enforcement: true
  credentials_visible_to_model: false
  recheck_before_write: true
  persist_execution_receipt: true
  reconcile_ambiguous_timeout_before_retry: true

verification:
  require_provider_readback: true
  track_external_observation_separately: true
  completion_requires_desired_state_evidence: true

```

DNS-specific safeguards

The proposed Domain Operations policy should:

- Preserve unrelated records and values.
- Show the complete affected record set before approval.
- Treat existing mail-routing and website records as protected by default.
- Require elevated review for nameserver, MX, DNSSEC, deletion, or whole-zone replacement operations.
- Re-read the affected records immediately before applying the approved change.
- Stop and request fresh approval when another actor changed the records.
- Verify provider acceptance separately from externally observable DNS results.
- Treat rollback as a controlled change—not blindly restore an old snapshot over newer legitimate updates.

For Route 53, enforce part of this boundary with IAM restrictions in addition to application-level checks. The provider supports record-name, record-type, and action conditions. ^[14]

Backend additions

Add these entities to the previous backend design:

```
connections
connection_capabilities
resource_inventory
state_snapshots
state_observations
desired_state_versions
gap_reports
change_plans
change_operations
approval_bindings
execution_receipts
verification_results
drift_events
```

The critical component is a server-side **policy enforcement gateway**. Neither the model nor an MCP server’s tool description should be the final authority on whether a production write is allowed.

5. Build this extension first

Implement one complete operational path before adding every tool:

1. **Connect Route 53:** verify account, permitted zone, read access, and limited write capability.
2. **Inspect:** collect provider records and identify the authoritative zone.
3. **Define desired state:** use an approved sending provider’s exact configuration requirements.
4. **Analyze:** produce an evidence-backed gap report and exact change proposal.
5. **Approve:** display account, zone, affected records, before/after values, and verification plan.
6. **Execute:** apply only the approved request through the policy gateway.
7. **Verify:** store provider receipts, re-read records, and track remaining verification.
8. **Expand:** add GoDaddy, then CRM and mailbox connections through Composio or tested adapters.

The first acceptance test should be:

“Enably connects to a permitted DNS zone, reads its state, identifies one missing approved record, proposes the exact change, applies it only after approval, and verifies the result—without modifying any unrelated record.”

That is the shift you are asking for: **Enably becomes a governed operator that understands current state, defines desired state, and closes the gap—not merely a collection of content-generation skills.**

✱

Show me how to map tool permissions and API scopes

Map permissions from **business task** → **agent** → **logical tool** → **provider API operation** → **required scope** → **allowed resource** → **approval rule**. Provider scopes define what your credentials can do; Enably’s backend policy must further restrict what each agent may do with those credentials.

For Enably, keep discovery read-only and route all DNS changes, CRM writes, and outbound sends through a separate, approval-enforcing execution gateway.

1. Build the permission matrix

Start with operations, not broad labels such as “AWS access” or “HubSpot access.”

Here is a proposed Enably mapping:

Task	Agent	Logical tool	Provider permission	Resource boundary	Approval
Inspect DNS records	State Discovery	<code>dns.records.read</code>	AWS <code>route53:ListResourceRecordSets</code>	Selected hosted zone	No per-call approval within authorized access
Inspect change status	Verification	<code>dns.change_status.read</code>	AWS <code>route53:GetChange</code>	Changes associated with the run	No
Add approved sending records	Domain Operations	<code>dns.records.apply</code>	AWS <code>route53:ChangeResourceRecordSets</code>	Selected zone, record names, types, and actions	Exact change batch
Read contacts	State Discovery	<code>crm.contacts.read</code>	HubSpot <code>crm.objects.contacts.read</code>	Connected account plus Enably-selected records	No per-call approval within authorized access
Update selected contact fields	Change Executor	<code>crm.contacts.update</code>	HubSpot <code>crm.objects.contacts.write</code>	Explicit contact IDs and allowed properties	Exact update payload
Generate social messages	Social DM Architect	<code>messages.draft</code>	No external API scope	Workspace assets	No external-action approval
Send outreach	Change Executor	<code>outreach.send</code>	Provider-specific sending permission	Approved sender and recipients	Exact recipients and content
Research public documentation	RAG DAL	<code>research.fetch</code>	Search/fetch provider authentication, if required	Approved public sources	Usually read-only

Route 53 supports fine-grained conditions for record names, types, and CREATE, UPSERT, or DELETE actions. HubSpot documents separate contact read and write scopes—but its contact write scope also permits deletion, so Enably must explicitly block deletion when only updates are intended. [\[28\]](#) [\[29\]](#)

Key distinction: an OAuth grant is not approval for a specific action. Connecting HubSpot with write access does not authorize an agent to update every contact.

2. Define a canonical contract

Keep your tool names stable even when the implementation changes from a direct API adapter to Composio or MCP.

This is a proposed Enably policy format—not an established native ROSTRv2 schema:

```
contract_version: enably.permissions.v1

defaults:
  decision: deny
  secrets_accessible_to_agents: false
  enforce_workspace_binding: true

tools:
  crm.contacts.read:
    provider: hubspot
    access: read
    required_scopes:
      - crm.objects.contacts.read

    permitted_agents:
      - state-discovery
      - gap-analyst

    resource_policy:
      account: connection.bound_account_id
      selection: run.selected_contact_ids

    data_policy:
      allowed_properties:
        - firstname
        - lastname
        - email
        - company
        - jobtitle
      redact_from_logs:
        - email

    approval:
      required: false

  crm.contacts.update:
    provider: hubspot
    access: write
    required_scopes:
      - crm.objects.contacts.write

    permitted_agents:
      - change-executor

    resource_policy:
      account: connection.bound_account_id
      selection: approval.target_contact_ids

    input_policy:
      allowed_properties:
        - jobtitle
      deny_operations:
        - delete
        - merge
        - bulk_update
```

```
approval:
  required: true
  bind:
    - workspace_id
    - connection_id
    - provider_account_id
    - target_contact_ids
    - exact_payload_hash
    - expires_at

execution:
  recheck_current_state: true
  audit_required: true
```

The backend must enforce this file. Loading it into an agent’s prompt alone is not a security boundary.

Store three separate permission records

Record	What it answers
Requested permissions	What did Enably ask the provider for?
Granted permissions	What did this specific connection actually receive?
Runtime authorization	What may this user and agent do in this workspace right now?

Execution should require all three to agree, plus action approval when applicable.

3. Apply provider-level restrictions

AWS: use IAM, not OAuth scopes

For a DNS pilot, use a dedicated role restricted to one hosted zone. Separate read access from narrowly bounded write access.

Example IAM permission policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadSelectedZone",
      "Effect": "Allow",
      "Action": "route53:ListResourceRecordSets",
      "Resource": "arn:aws:route53::hostedzone/Z_REPLACE_ME"
    },
    {
      "Sid": "ManageOnlyApprovedDkimRecords",
      "Effect": "Allow",
      "Action": "route53:ChangeResourceRecordSets",
      "Resource": "arn:aws:route53::hostedzone/Z_REPLACE_ME",
      "Condition": {
        "ForAllValues:StringEquals": {
          "route53:ChangeResourceRecordSetsNormalizedRecordNames": [
            "selector1._domainkey.example.com",
            "selector2._domainkey.example.com"
          ],
          "route53:ChangeResourceRecordSetsRecordTypes": [
            "CNAME"
          ],
          "route53:ChangeResourceRecordSetsActions": [
```

```

        "CREATE",
        "UPSERT"
    ]
}
}
}
]
}

```

Replace the example zone and record names with the actual approved resources. AWS documents these three condition keys specifically for restricting record names, record types, and change actions. ^[28]

This policy intentionally does not grant record deletion or arbitrary zone changes. It also does not constrain record values: Enably's approval gateway must enforce the exact approved values.

A role also needs a separately configured trust policy. For the initial pilot, supplying a known zone ID avoids granting unnecessary account-wide discovery access.

HubSpot: split discovery and execution

Use these capability profiles:

```

hubspot_discovery:
  scopes:
    - crm.objects.contacts.read
  enabled_tools:
    - crm.contacts.read

hubspot_contact_updates:
  scopes:
    - crm.objects.contacts.read
    - crm.objects.contacts.write
  enabled_tools:
    - crm.contacts.read
    - crm.contacts.update
  disabled_tools:
    - crm.contacts.delete
    - crm.contacts.merge

```

HubSpot scopes correspond to endpoint access; verify each endpoint's documented required scopes rather than inferring them from the object name. Account tier can also affect endpoint availability. ^[29]

Composio: scopes and tools are separate

Configure the desired OAuth scopes in a Composio auth config, then bind the resulting connected account to your Enably workspace and authenticated user. Composio auth configs define authentication method, requested scopes, and credentials; connected accounts hold the user-linked credentials. ^{[30] [31]}

Your local connection record could look like:

```

connection:
  id: enably_connection_123
  workspace_id: workspace_123
  owner_user_id: user_123

  provider: hubspot
  transport: composio

  composio_auth_config_id: "<returned-id>"

```

```

composio_connected_account_id: "<returned-id>"
verified_provider_account_id: "<verified-account-id>"

requested_scopes:
  - crm.objects.contacts.read

granted_scopes:
  - crm.objects.contacts.read

enabled_capabilities:
  - crm.contacts.read

status: verified

```

Changing a Composio auth config's scopes affects new connections; existing users must reconnect to receive the changed grant. Track connection-level granted scopes rather than treating the latest auth config as proof of permission.^[32]

For MCP adapters, apply the same rule: expose an allowlisted tool set and enforce authorization server-side. Do not allow a generic HTTP/proxy tool to bypass restrictions on specific tools.

4. Enforce permissions at execution

The gateway should use this decision order:

```

Authenticate user
→ Resolve workspace membership and role
→ Verify agent identity
→ Resolve workspace-bound connection
→ Verify provider account and granted permissions
→ Resolve provider operation from the tool registry
→ Validate target and input restrictions
→ Verify exact approval for writes
→ Recheck relevant current state
→ Execute through adapter
→ Store receipt and verification result

```

Illustrative server-side logic:

```

def authorize(request, policy, connection, approval):
    require_user_workspace_access(request)
    require_agent_allowed(request.agent_id, policy)
    require_connection_bound_to_workspace(connection, request.workspace_id)
    require_verified_provider_account(connection)
    require_granted_scopes(connection, policy.required_scopes)

    validate_resource_boundary(request, policy)
    validate_input_allowlist(request.payload, policy)

    if policy.approval_required:
        verify_approval_signature(approval)
        require_approval_not_expired(approval)
        require_approval_matches_exact_request(approval, request)
        require_approval_not_consumed(approval)
        require_current_state_matches_precondition(request)

    return "allowed"

```

For a DNS write, the approval should bind to:

```
approval_binding:
  workspace_id: workspace_123
  connection_id: aws_connection_123
  provider_account_id: "<verified-aws-account>"
  hosted_zone_id: "<resolved-zone>"
  operation: dns.records.apply
  exact_payload_hash: "<server-computed-hash>"
  before_state_hash: "<affected-records-hash>"
  expires_at: "<expiration>"
  single_use: true
```

The approval record must be created by the authenticated host workflow—not supplied or self-issued by the agent. Compute hashes from a consistent canonical representation.

If the DNS records change between inspection and execution, stop, regenerate the diff, and request fresh approval.

5. Test the boundaries

Before enabling production writes, test these cases:

Test	Expected result
Discovery agent attempts a DNS write	Denied
Executor has approval but lacks provider permission	Denied
Connected account belongs to another workspace	Denied
HubSpot write token used to delete a contact	Denied by Enably policy
Approved DNS request includes a different record value	Denied
Record changed after approval	Blocked pending fresh analysis
Approval expired or already consumed	Denied
Provider request times out after submission	Reconcile provider state before retrying
Valid approved write succeeds	Receipt stored; verification starts
Tool returns success but desired state is not verified	Mark as verification pending, not complete

For your first implementation, create **three files**: a tool permission manifest, a provider-permission mapping, and a connection capability registry. Together they make permissions inspectable: what the agent may request, what the credential can actually do, and what the backend will allow for the exact resource and action.



- 1. Enably-GTM-Project-Overview-2.docx
- 2. ENABLY-Product-Specification-Document.docx
- 3. Enably.xlsx
- 4. PRD.md
- 5. PROJECT_BRIEF.md
- 6. dev3/SKILL.md
- 7. Untitled-document.docx
- 8. KPI-Metric-Architect-GPT.docx
- 9. LinkedIn-Social-DM-Architect-GPT.docx

10. [rostr-orchestrator.md](#)
11. [rostr-agent-builder-tool-manifest.md](#)
12. <https://docs.composio.dev/docs/extending-sessions/custom-tools-and-toolkits>
13. https://docs.aws.amazon.com/Route53/latest/APIReference/API_ChangeResourceRecordSets.html
14. <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-sets-permissions.html>
15. <https://www.godaddy.com/resources/news/godaddy-dns-api-now-works-with-a-single-domain>
16. <https://developer.godaddy.com/en/docs/api-users>
17. <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/specifying-conditions-route53.html>
18. <https://aws.amazon.com/blogs/networking-and-content-delivery/implementing-fine-grained-amazon-route-53-access-using-aws-iam-condition-keys-part-1/>
19. <https://aws.amazon.com/blogs/networking-and-content-delivery/fine-grained-amazon-route-53-access-with-iam-condition-keys-part-3/>
20. <https://aws.amazon.com/blogs/networking-and-content-delivery/implementing-fine-grained-amazon-route-53-access-using-iam-condition-keys-part-2/>
21. <https://developer.godaddy.com/>
22. <https://composio.dev/toolkits>
23. <https://community.letsencrypt.org/t/godaddy-no-longer-allows-api-access-to-clients-e-g-for-dns-based-cert-renewal-if-you-have-less-than-50-domains/219377>
24. <https://composio.dev/>
25. <https://composio.dev/toolkits/agency/framework/autogen>
26. [dev3/ddc-planning-runtime.ts](#)
27. <https://developer.godaddy.com/en/docs/api-users/quickstart>
28. <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-sets-permissions.html>
29. <https://developers.hubspot.com/docs/apps/legacy-apps/authentication/scopes>
30. <https://docs.composio.dev/docs/tools-direct/authenticating-tools>
31. <https://docs.composio.dev/reference/api-reference/auth-configs>
32. <https://docs.composio.dev/docs/authentication/controlling-scopes>
33. <https://developers.hubspot.com/docs/api-reference/legacy/crm/activities/emails/update-email>
34. <https://docs.composio.dev/docs/authentication>
35. <https://github.com/ComposioHQ/composio/blob/next/docs/content/reference/authenticating-to-composio/project-api-key-permissions.mdx>
36. <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/specifying-conditions-route53.html>
37. <https://aws.amazon.com/blogs/networking-and-content-delivery/implementing-fine-grained-amazon-route-53-access-using-aws-iam-condition-keys-part-1/>
38. https://docs.aws.amazon.com/Route53/latest/APIReference/API_ChangeResourceRecordSets.html
39. <https://developers.hubspot.com/docs/apps/developer-platform/add-features/app-objects/reference>
40. <https://developers.hubspot.com/docs/api-reference/latest/crm/extensions/crm-cards/guide>
41. <https://developers.hubspot.com/docs/api-reference/latest/crm/objects/orders/get-order>
42. <https://developers.hubspot.com/docs/api-reference/legacy/crm/objects/contacts/create-contact>